

Active inference for event-triggered gait adaptation in quadrupedal robot locomotion

Abstract—Central pattern generators with attitude controllers produce signals that coordinate leg motion, allowing legged robots to walk robustly. Its parameters are tuned to the robot body and to the environment in which it operates. Whenever an event occurs that alters the robot’s dynamics, the generator parameters may not be optimal again. Under suboptimal parameters, the robot may become unstable and eventually fall. To prevent this, we propose an agent that triggers a parameter updating phase when an uncertainty-weighted prediction error crosses a threshold. The threshold is not tuned, but derived from a false positive rate. In the triggered phase, the agent will propose candidate sets that it believes will restore predictions of level forward motion and restore optimality before it falls. Planning and action is performed by expected free energy minimization. We perform simulated experiments with sudden payload shifts, comparing the proposed active inference agent with state-of-the-art baselines in terms of travel distance.

I. INTRODUCTION

Legged machines are increasingly asked to work for long uninterrupted bouts in non-stationary conditions. Events such as payload shifts and actuator damage can lead to stumbling and falling. Animals adapt to such events effortlessly and engineers have long pursued it by replicating their morphologies [1], [2]. Yet current leg controllers still fall far short. For example, Boston Dynamics’ Spot relies on centralized, sensor-intensive control that is effective but complex and costly [2]. Central pattern generators (CPGs) offer a simpler alternative. In vertebrates, these spinal circuits generate the periodic signals of movements such as walking, freeing higher centers to modulate speed and gait through simple inputs [3], [4], and their decentralized architecture lowers command latency and bandwidth [5]. The price is that the rhythm is shaped by only a handful of parameters, held fixed during walking, so a setting that is stable under one condition need not be stable under another. When an event changes the condition mid-bout, staying upright requires re-tuning those parameters online.

Coupled nonlinear oscillators are biologically plausible and computationally efficient [6], [5]: mutual inhibition synchronizes the joints into coordinated gaits [6], [7], [8], and their limit cycles recover quickly from perturbations [7]. An open-loop CPG walks stably on flat ground, but a change in condition induces body rotations the rhythm alone cannot correct, so feedback must close the loop: contact feedback resynchronizes the oscillators to the actual footfalls [8], and posture feedback corrects the trunk attitude. Virtual model control (VMC) computes joint actions as if springs and dampers were attached to the body [9]; such springs pull the trunk level over terrain that defeats an open-loop CPG

[10], and we adopt this attitude VMC as a reactive posture layer (Section II-B).

Reactive feedback leaves the parameters untouched; a large or persistent change requires adapting them. A damaged robot can search a precomputed behavioral repertoire by trial and error, recovering within a dozen physical trials [11], while rapid motor adaptation infers a latent encoding of the condition from recent proprioception and adjusts a learned policy in a fraction of a second [12]. Each incurs a cost we avoid: trial and error executes every candidate on the compromised robot, and the learned policy needs extensive training. A complementary line tunes gait parameters as online black-box optimization, fitting a Gaussian process surrogate to the parameter-score pairs seen so far and keeping proposals inside a trust region [13], but it too can only score a candidate by walking on it.

Deciding *when* to act is a control problem in its own right. Event-triggered control replaces a fixed update rate with a condition that fires once a monitored signal leaves a tolerance band, so effort is spent when the plant departs from expectation rather than on a schedule; paired with a Gaussian process, whose posterior variance reports where a model is still trustworthy, it has served remotely controlled vehicles for which every update is costly [14]. We inherit that structure but not its scarce resource: here physical trials are the constraint, not bandwidth, so the trigger gates a re-tuning whose cost would otherwise be paid in falls.

Both decisions follow from a single principle. Active inference casts perception and control as inference under one generative model, with preferred outcomes as a prior over observations and actions chosen by minimizing expected free energy (EFE) [15], [16]. That objective splits into a cross-entropy to the goal prior and an information gain [17], [18], so exploration follows from the agent’s own uncertainty rather than a separately tuned acquisition rule, and the prior that scores a candidate gait also reports when the current one has drifted.

We contribute a prediction-error trigger that decides *when* to re-tune the CPG, and a unified active inference agent that decides *what* to re-tune to. Screening candidates against a memory of failed gaits, it avoids the destabilizing on-robot trials that grid search, Bayesian optimization and extremum-seeking control require. Under a persistent payload shift it accrues roughly a quarter of their falls and detects each shift sooner.

II. PROBLEM STATEMENT

Let $\xi = (\alpha, \dot{\alpha}, p, \dot{p})$ represent the orientation $\alpha \in SO(3)$, angular velocity $\dot{\alpha}$, position $p \in \mathbb{R}^3$, and linear velocity $\dot{p}$ of the trunk of a quadrupedal robot in a world frame. The orientation α consists of roll α_r , pitch α_p and yaw α_y , and the position of a forward, a lateral and a vertical component (p_x, p_y, p_z) . Furthermore, let φ be the robot's joint angles, $\dot{\varphi}$ the change in joint angles and τ the torques. Each foot i additionally reports a binary contact state $c_i \in \{0, 1\}$. The map from $(\varphi, \dot{\varphi}, \tau)$ to ξ is unknown as it depends on the full rigid-body dynamics, the terrain and the switching foot-ground contact, none of which is available in closed form. Locomotion is then the problem of finding torques that drive the base toward a desired state, such as a target forward velocity with roll and pitch near zero.

A. Central pattern generator

A CPG is a dynamical system that converges to a limit cycle, absorbing disturbances such as missteps due to uneven terrain. We consider a CPG based on a network of four coupled Hopf oscillators [19]. For $i = 1, \dots, 4$, we define a Cartesian coordinate that evolves according to

$$\dot{z}_{1i} = \beta_1 (v - (z_{1i}^2 + z_{2i}^2)) z_{1i} - \omega_i z_{2i}, \quad (1)$$

$$\dot{z}_{2i} = \beta_2 (v - (z_{1i}^2 + z_{2i}^2)) z_{2i} + \omega_i z_{1i} + \gamma \sum_{j=1}^4 k_{ij} z_{2j} + s_i, \quad (2)$$

where the β are positive constants that control the convergence rate to the oscillator's limit cycle and v is the squared limit-cycle amplitude. Each Hopf oscillator has a stance and a swing phase that it balances through the parameter ω_i ;

$$\omega_i = \omega_{\text{stance}} \zeta(-bz_{2i}) + \omega_{\text{swing}} \zeta(bz_{2i}). \quad (3)$$

$\zeta(\cdot)$ interpolates between the two frequencies based on the sign and magnitude of z_{2i} , with b sharpening the balance. The $k_{ij} \in \{-1, 0, 1\}$ indicate inhibitory or excitatory coupling between legs i and j . The overall coupling gain γ has been shown to reduce the dependence on initial conditions [19].

The CPG incorporates a sensory feedback loop through s_i in Eq. 2 [8]. Leg i is classified as *swinging* (i.e., no contact) when it exceeds a small positive threshold $\bar{z}$ and as *standing* (i.e., with contact) when it is below a small negative one $\underline{z}$, with a hysteresis band in between;

$$\rho_i := \begin{cases} 0 & |z_{2i}| > \bar{z} \\ 1 & |z_{2i}| < \underline{z} \\ \rho_i^- & \text{otherwise} \end{cases} \quad (4)$$

where ρ_i^- is the previous classification, so that the description latches the phase and prevents rapid switching. Resynchronization of the swing versus stance phase is triggered by a mismatch between the oscillator's phase ρ_i and the measured foot contact c_i :

$$s_i = \begin{cases} \lambda (\omega_i z_{1i} - \gamma \sum_j k_{ij} z_{2j}) & \text{if } c_i = \rho_i \\ \eta \text{sign}(z_{2i}) & \text{if } c_i \neq \rho_i. \end{cases} \quad (5)$$

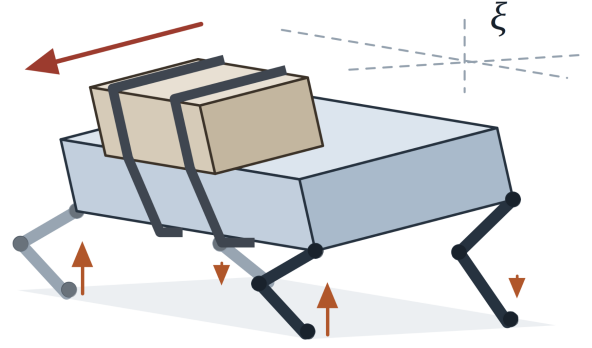


Fig. 1. Schematic of quadrupedal robot with base state ξ being perturbed by leg motion. Events such as payload shifts alter which parameters are optimal. Arrows at feet indicate compensation by virtual attitude controller.

We refer to λ as the phase match parameter and η the phase mismatch. During phase match, λ rescales the oscillator's own constants to $(1 + \lambda) \omega_i$ and $(1 - \lambda) \gamma$. During phase mismatch, z_{2i} is driven away from zero, i.e., the leg holds its current swing or stance until it reaches a match again.

The oscillator states (z_1, z_2) map to the joint angles φ through

$$\varphi_i^{\text{hip}} = \varphi_0^{\text{hip}} + A_{\text{hip}} z_{1i}, \quad (6)$$

$$\varphi_i^{\text{knee}} = \varphi_0^{\text{knee}} - A_{\text{knee}} \max(0, z_{2i}) + \Delta_i, \quad (7)$$

with φ_0^{hip} , φ_0^{knee} the angle origins and A_{hip} , A_{knee} the swing amplitudes. During swing ($z_{2i} > 0$) the knee flexes to lift the foot while the hip advances with z_{1i} , and during stance the knee holds its nominal extension. The term Δ_i is the posture correction of Section II-B. Overall, CPG parameters are collected into the vector

$$\theta = [\gamma, \omega_{\text{swing}}, \omega_{\text{stance}}, \eta, \lambda, A_{\text{hip}}, A_{\text{knee}}, b]. \quad (8)$$

B. Attitude virtual model control

The CPG naturally absorbs disturbances and returns to its limit cycle, but there is no explicit mechanism to ensure the robot base maintains level. We impose an attitude proportional-derivative (PD) controller to match the hip positions to a level reference plane, similar to a set of mass-spring-dampers on a car [9], [10]. For roll and pitch, we define:

$$c_r = -(\kappa_p^r \alpha_r + \kappa_d^r \dot{\alpha}_r), \quad (9)$$

$$c_p = -(\kappa_p^p (\alpha_p - \bar{\alpha}_p) + \kappa_d^p \dot{\alpha}_p), \quad (10)$$

with stiffness and damping gains κ . Roll is regulated towards zero to combat transient rocking. Pitch is regulated towards a slowly adapting baseline $\bar{\alpha}_p$, an exponential moving average over recent pitch values, so the sustained pitch of an incline is tolerated rather than fought. The commands distribute over the legs as:

$$\Delta_1 = \text{clip}(c_p + c_r, \pm \delta_{\text{max}}) \quad (11)$$

$$\Delta_2 = \text{clip}(c_p - c_r, \pm \delta_{\text{max}}) \quad (12)$$

$$\Delta_3 = \text{clip}(-c_p + c_r, \pm \delta_{\text{max}}) \quad (13)$$

$$\Delta_4 = \text{clip}(-c_p - c_r, \pm \delta_{\text{max}}), \quad (14)$$

where the index runs over front left, front right, back left and back right, and $\delta_{\max}$ bounds the correction. The knee angle sets the effective leg length, so the legs on a sinking side of the trunk extend to push that corner up while those on the rising side shorten.

C. Shifts in parameter optimality

Let e denote the *operating condition* of the robot in the world. It covers extrinsic factors such as terrain incline and friction, together with intrinsic ones such as the center of mass of the base. Running the controller with parameters θ under condition e , with the attitude feedback of Section II-B active, induces a distribution $p(\xi | \theta, e)$ over the base state. Our desired state is expressed as a goal distribution over ξ , parameterized as a Gaussian distribution:

$$p(\xi | \xi^*, \Sigma^*) = \mathcal{N}(\xi | \xi^*, \Sigma^*), \quad (15)$$

where ξ^* represents the target and Σ^* indicates permissible deviation around that target. Our locomotion objective is then written as a cross-entropy between the induced distribution and the goal distribution:

$$J(\theta, e) = \mathbb{E}_{p(\xi | \theta, e)} \left[-\ln p(\xi | \xi^*, \Sigma^*) \right], \quad (16)$$

In practice the expectation is a time average over an evaluation window on low-pass-filtered angles, so that the rhythmic rocking of the gait is not penalized. We can now define a set of CPG parameters that is optimal with respect to an operating condition e :

$$\theta^*(e) = \arg \min_{\theta} J(\theta, e). \quad (17)$$

Crucially, $\theta^*(e)$ is a function of the environment and the robot's body. Optimizing Eq. 16 under differing terrain, load or actuator torque yields distinct parameter vectors, each most stable for its tuned condition. Thus, a change in condition relocates the optimum, $\theta^*(e_{\text{old}}) \neq \theta^*(e_{\text{new}})$. Examples of such changes include the robot stepping onto a new surface, a payload that slides off or an actuator that short-circuits (Figure 1).

Because θ is held fixed, a shift leaves the controller suboptimal and the attitude deviations compound: as the trunk tilts, recovery soon demands more than the bounded feedback of Eq. 14 can supply, and locomotion fails. Resuming stable walking therefore requires two decisions, *when* the optimum has moved and *what* to re-tune θ toward. This has to be done quickly enough to prevent instabilities from increasing.

III. ACTIVE INFERENCE AGENT

The active inference agent consists of a probabilistic generative model that predicts observations, updates a probability distribution over parameters after each observation, and plans actions to reach a target based on expected free energy minimization [18]. The agent makes a noisy observation $\mathbf{y}_k$ of the robot state ξ_k at time k , and acts by modulating the CPG parameter vector θ . While the agent is event-driven, the CPG is always running and generates joint angle setpoints which are turned to servo torques by a PD controller (see Figure 2).

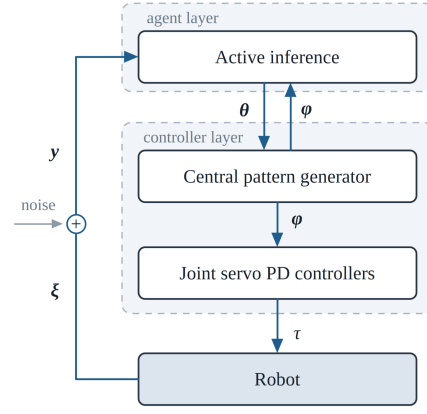


Fig. 2. Overview of information flow: the agent observes base state and generates CPG parameters, which are turned into oscillator states and joint angle setpoints. These are picked up by a PD controller and turned into torques. Torques are based to the servomotors and the robot changes base state. The PD controller operates fastest, then the CPG and lastly the agent.

A. Probabilistic model and belief updating

We propose a probabilistic model based on multivariate autoregression with exogenous input (MARX). Let $\bar{\mathbf{y}}_k = [\mathbf{y}_{k-1}, \dots, \mathbf{y}_{k-m_y}]$ be the recent history of base states. The evolution of $\mathbf{y}_k$ is described as a Gaussian distribution over autoregressive coefficients A and process noise precision W :

$$p(\mathbf{y}_k | \bar{\mathbf{y}}_k, \theta, A, W) = \mathcal{N}(\mathbf{y}_k | A^\top \begin{bmatrix} \bar{\mathbf{y}}_k \\ \phi_k \end{bmatrix} + g(\theta), W^{-1}). \quad (18)$$

The joint angles are incorporated as an exogenous signal. The function $g(\cdot)$ stems from a set of Gaussian process regression models that map the CPG parameters θ to each of the base state dimensions. It is fitted separately to a data set $\{(\theta_j, \mathbf{y}_j)\}_{j=1}^N$ of the gaits tried and the outputs they produced. As such, it constitutes a memory that will prevent a repeated use of θ 's that lead to a fall. As the GP models are trained separately, at any tick of the clock, their prediction constitutes a Gaussian distributed random variable, which is fused with the base autoregression term to form the mean of the total autoregressive likelihood.

A matrix-normal-Wishart posterior over the coefficients A and the noise precision W is updated in closed form at every control step [20]. This produces a multivariate location-scale T-distributed posterior predictive distribution:

$$p(\mathbf{y}_k | \bar{\mathbf{y}}_k, \theta) = \mathcal{T}_{v_k}(\mathbf{y}_k | \mu_k(\theta), \Sigma_k(\theta)), \quad (19)$$

where v_k is a degrees-of-freedom parameter.

B. Planning and inferring action

When adaptation is triggered (Section III-C), the agent calculates the Expected Free Energy of its predicted output under the goal prior [18],

$$G(\theta) = \underbrace{\frac{1}{2} (\xi^* - \mu_k(\theta))^\top \Sigma^{*-1} (\xi^* - \mu_k(\theta))}_{\text{goal-seeking}} + \frac{1}{2} \text{tr}(\Sigma^{*-1} \Sigma_k(\theta)) - \underbrace{\frac{1}{2} \ln |I + \Sigma_k(\theta) \Sigma^{*-1}|}_{\text{information-seeking}}, \quad (20)$$

The goal-seeking term seeks to match the mean and variance of the predictive distribution to the goal distribution. The information gain term favours elements of θ for which the predictive variance is larger than the goal variance as those represent epistemic uncertainty to be reduced. The agent will select the action that minimizes EFE:

$$\hat{\theta} = \underset{\theta}{\operatorname{argmin}} G(\theta) \quad (21)$$

Safety is not a separate constraint but a consequence of the goal prior. A gait that the memory associates with falling predicts outputs far from ξ^* , which inflates the goal-seeking term and is avoided while the agent is still exploring.

C. Triggering events based on prediction error

Re-tuning θ at every step is wasteful and, as Section IV shows, actively destabilizing, so the agent adapts only when its own predictions leave the goal. The perturbations it must survive threaten the *uprightness* of the trunk rather than its speed: a shift that a slower gait can absorb is tolerable, whereas a tilting trunk is not. The trigger therefore reads the attitude marginal of the goal prior of Eq. 15, namely the cross-entropy of the one-step predicted pitch and roll ($\mu_k^\alpha, \Sigma_k^\alpha$) to their level target $\alpha^* = 0$,

$$\begin{aligned} \varepsilon_k &= H(\mathcal{N}(\mu_k^\alpha, \Sigma_k^\alpha) \| p(\alpha | \alpha^*, \Sigma_\alpha^*)) \\ &= \frac{1}{2} \mu_k^{\alpha^\top} \Sigma_\alpha^{*-1} \mu_k^\alpha + \frac{1}{2} \operatorname{tr}(\Sigma_\alpha^{*-1} \Sigma_k^\alpha) + c, \end{aligned} \quad (22)$$

the same cross-entropy-to-goal that Eq. 20 minimizes, restricted to the attitude subspace and evaluated on the belief rather than on a candidate gait. It is small while the trunk is predicted level and grows when a perturbation drives it to tilt, requiring no model of the perturbation and no signal of when it occurs. Standardizing ε_k against a healthy warm-up baseline $(\bar{\varepsilon}, s)$ and accumulating the excess in a one-sided CUSUM,

$$S_k = \max(0, S_{k-1} + (\varepsilon_k - \bar{\varepsilon})/s - \kappa), \quad (23)$$

fires an event when S_k exceeds a bound h , the slack κ being the per-step surprise tolerated as noise. Because the CUSUM integrates over time, it registers the sustained mismatch that a perturbation creates while rejecting the stride-frequency ripple that dominates a one-step residual at 100 Hz.

Event logic keeps a single perturbation from drawing more than a single correction. The monitor is armed only after a sustained healthy interval past a grace period, so the transient following a recovery is never mistaken for a fresh perturbation. Once a gait is applied, the accumulator is reset and held at zero for a settle window covering the ramp-in and one scoring hold, after which the monitor latches for the remainder of the event. Reading uprightness alone is what lets the trigger *quiet*: once the robot is upright again it falls silent, even under a recovered gait that walks slower or must crab sideways to absorb an asymmetric fault. The agent thus stops adapting when it has restored balance, rather than chasing a nominal speed it may no longer reach. The forward-velocity preference is not discarded, since it stays in the control objective of Eq. 20, which keeps seeking

speed among the gaits that hold the trunk level. Sensing and control therefore read one goal prior in two ways: its attitude marginal decides *when* to act, and the full prior decides *what* to do.

The bound h in Eq. 23 need not be tuned, because the accumulator carries units. Since ε_k is a cross-entropy in nats, sS_k is the free energy forgone, in nats, by holding the incumbent gait since the excursion began, so h is a *price* rather than a sensitivity. Adapting carries a price of its own: writing $p(\theta) = \mathcal{N}(\theta_k, \Upsilon^{-1})$ for a prior over gait parameters centered on the incumbent, a switch to $\theta_k + \Delta\theta$ costs

$$C(\Delta\theta) = -\ln p(\theta_k + \Delta\theta) + c = \frac{1}{2} \Delta\theta^\top \Upsilon \Delta\theta, \quad (24)$$

paid once at the switch, whereas ε_k is a rate incurred afresh at every step the incumbent is held. A threshold on ε_k itself thus compares a rate to a one-time cost, and is either tripped by transients or, set honestly, silent until the robot is already failing. Accumulating first removes the inconsistency, and the bound becomes the price,

$$h = C(\Delta\theta)/s, \quad (25)$$

turning Eq. 23 into a payback test: the agent switches once the value forgone by not switching repays what switching costs, after a delay $h/(\bar{\varepsilon} - \kappa)$ that shortens as the standardized excess $\bar{\varepsilon}$ grows.

Two caveats limit how literally Eq. 25 may be read: it prices only the subspace the gait parameters can steer, so it is a *lower* bound on the principled threshold, and it is only as meaningful as Υ . At the operating point of Section IV-A, $h = 5$ prices a switch at 0.13 nats under the measured $s = 0.025$ nats, whereas the gaits the agent actually applies lie some twelve prior standard deviations out; pricing that realized move would delay the trigger by a median of 10.8 s and silence it in most bouts. The realized move is large because the search explores, not because correction demands it, and pricing instead the correction that the belief itself deems necessary, through its control gain, is left to future work.

Calibration in the experiments below therefore rests on the slack κ , the only quantity still set by hand. Over the 150 s of settled nominal walking the standardized excess peaks at 2.69, so at $\kappa = 3$ every increment of Eq. 23 is negative; the accumulator is pinned at zero by induction and no false alarm is possible on that sample, whatever h is. The bound h then only imposes a short persistence requirement once a perturbation lifts the excess past the slack. Beyond the observed support the guarantee weakens from certainty to an exponential rate, and h is what sets it.

Proposition 1. Write $e_k = (\varepsilon_k - \bar{\varepsilon})/s$ for the standardized excess and $T = \inf\{k : S_k > h\}$ for the firing time. Suppose, conditional on the warm-up estimates $(\bar{\varepsilon}, s)$, that the increments $e_k - \kappa$ are independent and identically distributed with negative mean, and that some $\theta^* > 0$ solves $\mathbb{E}[e^{\theta^*(e_k - \kappa)}] = 1$. Then an unperturbed run of unbounded length fires with probability at most

$$\Pr(T < \infty) \leq e^{-\theta^* h}, \quad (26)$$

so a false-alarm budget a is met by any $h \geq \ln(1/a)/\theta^*$.

The proof is given in the Appendix. The threshold is thus fixed twice over, and not by a sensitivity dial: Eq. 25 prices it against the cost of switching, and Eq. 26 prices it against the false alarms one is willing to tolerate. The two agree at the operating point of Section IV-A. For Gaussian e_k one has $\theta^* = 2(\kappa - \mu)/\sigma^2$, giving $e^{-2\kappa h} = 9 \times 10^{-14}$ at the standardized values. That is optimistic, since ϵ_k is strongly autocorrelated at 100 Hz, with a lag-one correlation of 0.92 and a long-run variance 8.9 times the marginal one; substituting the long-run variance for the one-step generating function gives $\theta^* \approx 2.7$ instead. A budget of $a = 10^{-6}$ then demands $h \geq 5.1$, within a few percent of the $h = 5$ that the switching price sets independently. The transient following a recovery reaches several hundred standard deviations and is kept from firing by the arming logic of Section III-C rather than by κ .

No matching guarantee is claimed against missed detections. It would require the post-change increments to have positive mean, whereas the payload shift of Section IV-A raises the mean standardized excess only to 0.93 against $\kappa = 3$, so the accumulator still drifts downward on average and fires through excursions of a heavy-tailed statistic rather than through drift. This suffices in practice, since every event in Section IV-A is detected, but we treat the false-negative rate as an empirical matter throughout.

IV. EXPERIMENTS

We evaluate the unified active inference agent on the Laikago quadruped in PyBullet, under a perturbation of the robot's own dynamics: a shifting trunk payload. The agent runs with the attitude controller of Section II-B active and adapts only the high-leverage subset of the parameter vector θ of Eq. 8, within fixed bounds throughout, while adaptation is gated by the prediction-error trigger of Section III-C. Holding the incumbent gait $\theta^*(\text{flat})$ without adapting is the reference against which the agent is measured. Control runs at $\Delta t = 0.01$ s with a target forward velocity of 0.5 m/s. Each perturbation event is scored by the criterion of Eq. 16 over a 10 s hold window with the ramp-in excluded, and gait switches are ramped in over 0.3 s so that the gait never jumps. We report that criterion on the attitude marginal of the goal prior and the forward-speed term separately, as distance covered. The two are reported apart because no gait in the searched subset advances under this payload, the clairvoyant oracle included: with the speed term folded in, the criterion is nine parts speed deficit and ranks the oracle last, which measures the perturbation rather than the adaptation.

Adaptation runs asynchronously with the simulation, as it would on hardware. The controller steps the physics at 100 Hz on one thread while each gait proposal, the Expected Free Energy minimization of Eq. 20 with its Gaussian process fit included, is computed concurrently off the control loop. A proposed gait takes effect only once that computation returns, so the wall-clock cost of the optimizer appears as a control latency over which the robot keeps walking on the unadapted gait, rather than as a frozen clock. Each bout being

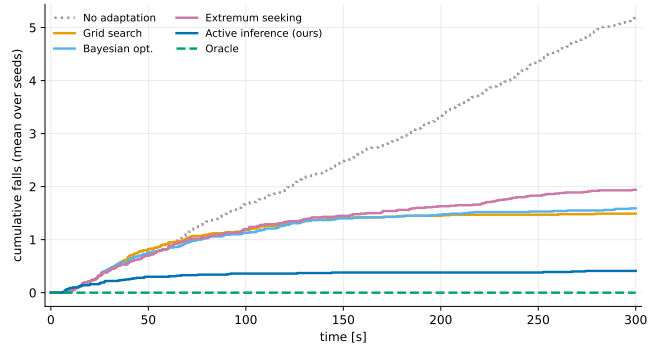


Fig. 3. Cumulative falls over a 300 s continual bout under the persistent payload shift (mean over one hundred seeds). Holding the incumbent gait accrues falls throughout. The three arms that must trial a candidate on the loaded robot (grid search, Bayesian optimization, extremum seeking) find a tolerant gait early but pay for the search first. Our agent, which screens each candidate against a memory of past falls before committing to it, falls far less.

paced to real time, a slower optimizer is charged honestly, through the extra ground covered before its gait engages, and we report this compute latency alongside the detection latency below.

A. Adapting to payload shifts

The scenario perturbs the mass distribution of the robot rather than its actuation. The Laikago carries an 8 kg payload rigidly attached above its trunk, whose own mass is 10 kg, and at random intervals the payload shifts off the sagittal plane, 0.215 m laterally and 0.20 m to the rear, ramped over 1 s so that the change is a sustained bias rather than an impulse. The offset then persists: once engaged it is held indefinitely, so the trunk bias compounds until the robot either falls or settles into a gait that tolerates the load. This offset sits at the edge of recoverability. Holding the incumbent gait $\theta^*(\text{flat})$, the robot tips over on 85% of shifts after a median of 33 s under the load, yet a re-tuned gait can stay upright indefinitely, so the perturbation rewards adaptation without being hopeless. The bout is non-episodic and runs for 300 s: whenever the robot falls it is stood upright and the payload recentered, and after 2 to 8 s the shift recurs. A controller that never adapts therefore meets the same perturbation over and over, whereas one that finds a tolerant gait simply holds it and walks on, so an agent that remembers earlier outcomes should stop repeating the parameter choices that fell.

At each detected shift the agent proposes a gait by minimizing the Expected Free Energy of Eq. 20 over its control map, favoring parameters whose predicted outputs lie near the goal while avoiding the regions its memory associates with falling. It searches only the high-leverage subset $\{\gamma, \omega_{\text{swing}}, \eta, \lambda, b\}$ of Eq. 8, and the memory persists across recurrences within the trial, so the map fills in as the shift repeats. Adaptation is triggered by the cross-entropy criterion of Section III-C: the MARX dynamics belief, updated online from the trunk outputs and the measured joint angles, drives the CUSUM once its one-step prediction departs the goal,

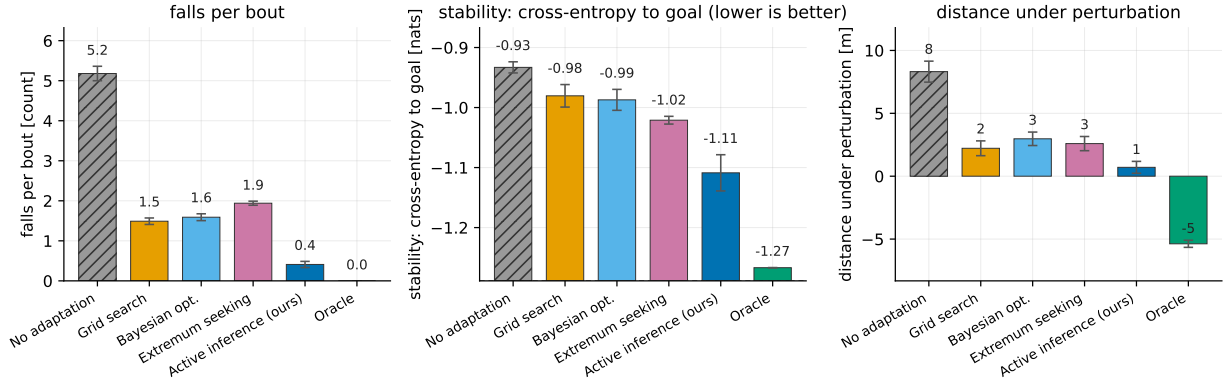


Fig. 4. Per-method outcome under the payload shift (mean over one hundred 300 s bouts): falls per bout, the criterion of Eq. 16 on its attitude marginal, and distance traveled while the payload is displaced. Bars in the middle panel rise from -1.29 nats, the value a trunk held exactly at the goal would score, so bar length is the cross-entropy in excess of that floor and shorter is better. The criterion separates the arms without counting a single fall, charging instead the tilting that precedes one. The adapting arms nonetheless cover less ground than the fixed gait, and the clairvoyant oracle is the most retrograde of all, which shows that the backward drift is a property of the gait parameterization rather than of the search.

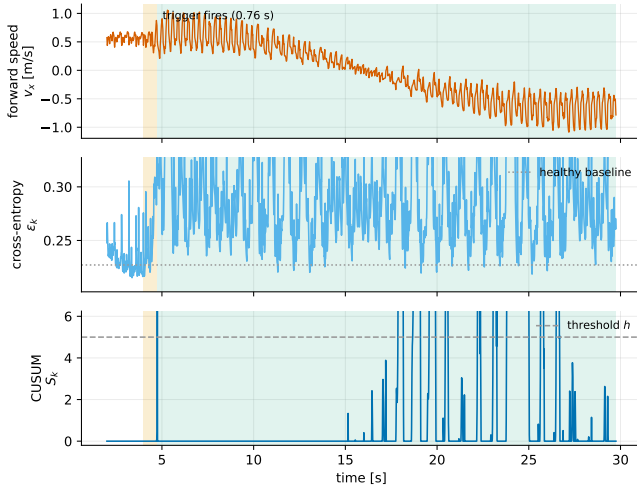


Fig. 5. The uprightness-only trigger over one payload shift (ours). *Top*: forward speed, which the tolerant gait carries slowly backward under the load. *Middle*: the attitude cross-entropy E_k rests at its healthy baseline and rises when the shift tilts the trunk (shaded orange until the trigger fires, green once the proposed gait is active); it settles above baseline rather than returning to it. *Bottom*: the CUSUM S_k crosses its threshold h within 0.76 s, is held at zero through the settle window, and re-crosses later in the bout without drawing a second adaptation, the monitor having latched for the event.

and re-arms once the robot has recovered after each recenter.

Four further arms, each searching that same subset so that the comparison is head to head, place the agent among the alternatives. Three are online optimizers that can judge a candidate gait only by running it: Latin-hypercube sampling (grid search), online Bayesian optimization, and extremum-seeking control, the classical model-free tuner, which dithers each parameter at a distinct frequency and demodulates the event score into a gradient estimate [21]. Extremum seeking is the sharpest form of the difficulty, since even its exploratory dither must be executed on the already-loaded robot. The fourth is a clairvoyant oracle that jumps straight to a pre-fit tolerant gait, an upper anchor on any parameter

switch.

Because a tolerant gait, once found, simply keeps walking, we run one hundred 300 s bouts per method and report the *cumulative* falls that each accrues over a bout rather than a per-event rate (Fig. 3). Holding the incumbent, the robot falls 5.18 ± 0.18 times per bout (mean $\pm$ s.e.m.), never adapting and tipping over on 85% of recurrences. Our agent instead falls only 0.41 ± 0.08 times, finding a tolerant gait after fewer than one fall on average and within the first 20 s, after which it holds that gait for the rest of the bout. The three online optimizers fall in between, at 1.49 ± 0.08 (grid search), 1.59 ± 0.09 (Bayesian optimization) and 1.94 ± 0.05 (extremum seeking) falls per bout, each significantly worse than our agent (Mann-Whitney on per-seed fall counts, $p < 10^{-26}$). They fall more because each can assess a candidate only by executing it on the already-shifted robot, where a poor candidate tips it over rather than merely scoring badly. Extremum seeking is worst of the three, significantly worse than both grid search ($p \approx 6 \times 10^{-9}$) and Bayesian optimization ($p \approx 3 \times 10^{-6}$), even though its proposal is the cheapest to compute: its dither is by construction a perturbation applied to the loaded robot, so the exploration it needs in order to estimate a gradient is paid for in falls. A clairvoyant oracle never falls, confirming that a recoverable gait exists within the searched subset.

Triggering and proposing are both prompt enough to run online. The goal-prior trigger detects the shift in 0.93 s on average, against 1.26 to 1.38 s for the prediction-error monitors driving the other arms, with no false alarms in any arm. Under the asynchronous protocol the gait proposal costs a median of one control step, since a tolerant gait is usually found before any Gaussian process fit is needed, rising to 1.74 s in the tail once the map is being fit.

The agent also stops re-tuning once a tolerant gait is in place, adapting exactly once per shift event across all one hundred bouts, so it does not chatter (Fig. 5). Two measurements qualify what produces that restraint. The attitude prediction-error does not return to its healthy baseline: it

settles about 17% above the pre-shift level, and comes within 10% of baseline in only 5% of bouts, since a load held off-center is never quite free. The accumulator does re-cross its threshold later in most bouts, and no second adaptation follows because the monitor latches for the remainder of the event. The restraint is therefore a property of the event logic rather than evidence that the residual mismatch has vanished, and whether re-tuning on that residual would help is untested.

Scored by the criterion itself, much the same ordering appears without a fall ever being counted (Fig. 4). Our agent reaches -1.11 ± 0.03 nats against -0.93 ± 0.01 for the fixed gait, the three optimizers falling between at -0.98 (grid search), -0.99 (Bayesian optimization) and -1.02 (extremum seeking), and the oracle at -1.27 , against a floor of -1.29 for a trunk held exactly level. The margin over each optimizer is significant (Mann-Whitney on per-seed means, $p < 10^{-11}$), whereas the three do not order among themselves as their fall counts do: extremum seeking scores best of them here while falling most, the criterion charging the tilt that precedes a fall rather than the fall itself. What the criterion adds is therefore corroboration from the prediction-error side, that the arms which fall less are also the ones held closer to the goal throughout.

At this offset the tolerant gaits are more compromised than the fall counts alone convey (Fig. 4). Every adapting arm settles into a gait that walks slowly *backward* under the load, at -0.12 m/s for our agent and -0.14 m/s for the three optimizers, against a 0.5 m/s forward target. The clairvoyant oracle is the most retrograde of all at -0.22 m/s, which is the informative part: the pre-fit gait that never falls is itself a backward one, so within this parameter subset no gait both tolerates the offset and makes forward progress. The benefit here is thus strictly falls avoided, and reaching a gait that walks forward under the load is capped by the gait parameterization, as we discuss next.

V. DISCUSSION

Our results separate two aspects of online adaptation that are easily conflated: deciding *when* to change the gait and deciding *what* to change it to. The prediction-error trigger resolves the first cheaply, reading uprightness rather than speed and re-arming to catch a recurring sequence, without a model of the perturbation. The second is where the danger lies, since grid search, Bayesian optimization and extremum seeking can judge a candidate gait only by walking on it: on an already-perturbed robot a poor candidate does not merely score badly, it tips the robot over. Extremum seeking makes the point most sharply, its dither being a deliberate perturbation of the loaded robot that costs more falls than the two search methods it is cheaper than. Our agent instead screens each candidate against a learned map with an explicit memory of past falls, and it does so even though the CPG is already stabilized by the attitude controller, so the benefit is complementary to the reactive posture loop rather than a substitute for it.

The unified formulation is, besides, an architectural economy: one goal prior specifies both when to act and what

to do, so the trigger needs no separately tuned detector, and reading uprightness off that prior detects the shift sooner than the hand-tuned monitors driving the other arms. We report this as a simplification rather than as a performance result, since these experiments do not separate how much of the reduction in falls is owed to the unified objective and how much to the memory screening alone.

Several limitations qualify these conclusions. The evaluation is in simulation, over a single intrinsic perturbation and a modest number of recurrences, and the reference optima are computed offline. More fundamentally, how much adaptation can recover is capped by the gait parameterization itself: under this payload offset no gait in the searched subset both tolerates the load and advances, the clairvoyant oracle itself walking backward at 0.22 m/s, so every online method settles somewhere between it and a standstill. Staying upright and making progress are thus in tension here, and our agent, like the others, resolves that tension toward staying upright. All arms pay the same price, so the comparison stays clean, but adapting *without* surrendering the locomotion objective is left open, and likely requires a richer control representation rather than a better search within this one. Validating on hardware, and across a wider range of perturbations, are natural next steps.

VI. CONCLUSIONS

We augmented a CPG of coupled Hopf oscillators with an attitude virtual model controller and studied when and how to re-tune its gait parameters online as the operating condition changes. A prediction-error trigger decides when to adapt from a sustained loss of performance, and a unified active inference agent decides what to change by selecting parameters from a learned model of the body's response, with a persistent memory of past falls, rather than by physically trialing candidates. On a shifting trunk payload, the agent learns within a single bout to stop selecting the gaits that fall, recovering without the destabilizing trials that grid search, Bayesian optimization and extremum seeking require. A single goal prior serves both decisions, so no separately tuned detector is required. How far this recovery generalizes, and how much the gait parameterization itself caps it, remain open.

APPENDIX

PROOF OF PROPOSITION 1

Let $X_k = e_k - \kappa$, so that Eq. 23 reads $S_k = \max(0, S_{k-1} + X_k)$ from $S_0 = 0$, and let $R_n = X_1 + \dots + X_n$ sum the same increments without the floor at zero. Unrolling the recursion gives $S_n = \max_{0 \leq j \leq n} (R_n - R_j)$, the climb since the walk's lowest point: clipping at zero forgets everything before the last dip. The increments being i.i.d., reversing their order changes nothing, so that climb has the law of the walk's highest point in n steps. Firing is therefore the event that the walk ever gets h above its start.

The increments have negative mean, so a climb to h is a run of luck. Rescale the walk as $Z_n = e^{\theta^* R_n}$: the condition $\mathbb{E}[e^{\theta^* X_k}] = 1$ makes every step multiply Z by a

factor averaging one, so Z is a fair game worth 1 on average, and stays fair if stopped at the first time τ the walk exceeds h . Stopped there it is worth at least e^{θ^*h} , and something averaging one reaches that value only that rarely,

$$1 = \mathbb{E}[Z_{\tau \wedge n}] \geq e^{\theta^*h} \Pr(\tau \leq n).$$

Letting n grow gives Eq. 26, and solving $e^{-\theta^*h} \leq a$ gives the budget form. ■

REFERENCES

- [1] K. Y. Pettersen, “Snake robots,” *Annual Reviews in Control*, vol. 44, pp. 19–44, 2017.
- [2] R. Dwivedula, S. Modak, A. Akella, J. Biswas, D. Kim, and C. J. Rossbach, “Configbot: Adaptive resource allocation for robot applications in dynamic environments,” *arXiv preprint arXiv:2501.10513*, 2025.
- [3] M. Gouilding, “Circuits controlling vertebrate locomotion: moving in a new direction,” *Nature Reviews Neuroscience*, vol. 10, no. 7, pp. 507–518, 2009.
- [4] M. H. Dickinson, C. T. Farley, R. J. Full, M. A. R. Koehl, R. Kram, and S. Lehman, “How animals move: An integrative view,” *Science*, vol. 288, no. 5463, pp. 100–106, 2000.
- [5] A. J. Ijspeert, “Special issue: Central pattern generators for locomotion control in animals and robots: A review,” *Neural Networks*, vol. 21, no. 4, pp. 642–653, 2008.
- [6] J. Collins and S. A. Richmond, “Hard-wired central pattern generators for quadrupedal locomotion,” *Biological Cybernetics*, vol. 71, no. 5, pp. 375–385, 1994.
- [7] A. Sprowitz, L. Righetti, and A. J. Ijspeert, “Passive compliant quadruped robot using central pattern generators for locomotion control,” in *IEEE RAS & EMBS International Conference on Biomedical Robotics and Biomechatronics*, 2008, pp. 710–715.
- [8] L. Righetti and A. J. Ijspeert, “Pattern generators with sensory feedback for the control of quadruped locomotion,” in *IEEE International Conference on Robotics and Automation*, 2008, pp. 819–824.
- [9] J. Pratt, C.-M. Chew, A. Torres, P. Dilworth, and G. Pratt, “Virtual model control: An intuitive approach for bipedal locomotion,” *International Journal of Robotics Research*, vol. 20, no. 2, pp. 129–143, 2001.
- [10] M. Ajallooeian, S. Pouya, A. Sproewitz, and A. J. Ijspeert, “Central pattern generators augmented with virtual model control for quadruped rough terrain locomotion,” in *IEEE International Conference on Robotics and Automation*. IEEE, 2013, pp. 3321–3328.
- [11] A. Cully, J. Clune, D. Tarapore, and J.-B. Mouret, “Robots that can adapt like animals,” *Nature*, vol. 521, no. 7553, pp. 503–507, 2015.
- [12] A. Kumar, Z. Fu, D. Pathak, and J. Malik, “RMA: Rapid motor adaptation for legged robots,” in *Robotics: Science and Systems*, 2021.
- [13] D. Eriksson, M. Pearce, J. Gardner, R. D. Turner, and M. Poloczek, “Scalable global optimization via local Bayesian optimization,” *Advances in Neural Information Processing Systems*, vol. 32, 2019.
- [14] D. Jang and J. Yoo, “Event-triggered control with reliable Gaussian process learning for remote UAV control,” *Control Engineering Practice*, vol. 165, p. 106576, 2025.
- [15] K. Friston, F. Rigoli, D. Ognibene, C. Mathys, T. Fitzgerald, and G. Pezzulo, “Active inference and epistemic value,” *Cognitive neuroscience*, vol. 6, no. 4, pp. 187–214, 2015.
- [16] L. Da Costa, T. Parr, N. Sajid, S. Veselic, V. Neacsu, and K. Friston, “Active inference on discrete state-spaces: A synthesis,” *Journal of Mathematical Psychology*, vol. 99, p. 102447, 2020.
- [17] M. T. Koudahl, W. M. Kouw, and B. de Vries, “On epistemics in expected free energy for linear Gaussian state space models,” *Entropy*, vol. 23, no. 12, p. 1565, 2021.
- [18] W. M. Kouw, “Information-seeking polynomial NARX model-predictive control through expected free energy minimization,” *IEEE Control Systems Letters*, 2023.
- [19] S. Rutishauser, A. Sprowitz, L. Righetti, and A. J. Ijspeert, “Passive compliant quadruped robot using central pattern generators for locomotion control,” in *IEEE RAS & EMBS International Conference on Biomedical Robotics and Biomechatronics*, 2008, pp. 710–715.
- [20] T. N. Nisslbeck and W. M. Kouw, “Factor graph-based online Bayesian identification and component evaluation for multivariate autoregressive exogenous input models,” *Entropy*, vol. 27, no. 7, p. 679, 2025.
- [21] N. J. Killingsworth and M. Krstic, “PID tuning using extremum seeking: online, model-free performance optimization,” *IEEE Control Systems Magazine*, vol. 26, no. 1, pp. 70–79, 2006.